

Lecture 11 优化算法

绳伟光

上海交通大学微纳电子学系

2025-11-25



提纲

1. 现代优化算法简介

2. 线性规划算法

2.1 线性规划算法简介

2.2 规划算法的专业相关应用及其求解器

3. 模拟退火算法

3.1 模拟退火算法求解旅行商问题

3.2 模拟退火算法求解套裁问题

3.3 模拟退火的专业相关应用 (以 Placement 为例)

4. 遗传算法

4.1 遗传算法数值优化

4.2 遗传算法组合优化之旅行商问题

4.3 遗传算法的专业相关应用

5. 粒子群算法

5.1 粒子群算法用于数值优化

5.2 粒子群算法的专业相关应用



提纲

1. 现代优化算法简介

2. 线性规划算法

2.1 线性规划算法简介

2.2 规划算法的专业相关应用及其求解器

3. 模拟退火算法

3.1 模拟退火算法求解旅行商问题

3.2 模拟退火算法求解套裁问题

3.3 模拟退火的专业相关应用 (以 Placement 为例)

4. 遗传算法

4.1 遗传算法数值优化

4.2 遗传算法组合优化之旅行商问题

4.3 遗传算法的专业相关应用

5. 粒子群算法

5.1 粒子群算法用于数值优化

5.2 粒子群算法的专业相关应用



优化算法的概念

优化问题

优化问题就是具有多个可行解，可从中求出具有最大 (最小) 解的问题。从数学上看，该类问题的可行解分布在一定的解空间中，需要从解空间中找到具有最大值 (最小值) 的点

优化问题一般形式

一般优化问题具有如下形式：

$$\begin{aligned} & \min f(X) \\ & \text{s.t. } g(X) \geq 0, X \in D \end{aligned}$$

其中， $X = (x_1, x_2, \dots, x_n)$ 是一个向量， $f(X)$ 为目标函数； $g(X)$ 为约束方程； D 为定义域

定义域 D 可能定义了一个不规则多维复杂空间，如果该空间是凸的，且解可以是浮点数，则可使用线性规划算法求解；如果空间是凹的，或者限制解必须是整数，则求解非常困难！



优化算法

传统优化方法主要指梯度下降法、动态规划法、线性规划法等，这些算法一般出现的较早。现代优化算法主要指与传统优化方法相比出现较晚，于 20 世纪 80 年代以来兴起的一系列具有全局优化能力的优化算法^[1]，包括：

- 模拟退火
- 遗传算法
- 神经网络
- 粒子群算法
- 蚁群算法

现代优化算法一般用于求解难以找到高效解法的 NPC 问题！

[1] 王凌. 智能优化算法及其应用. 北京: 清华大学 & Springer 出版社, 2001.



优化算法的新方向

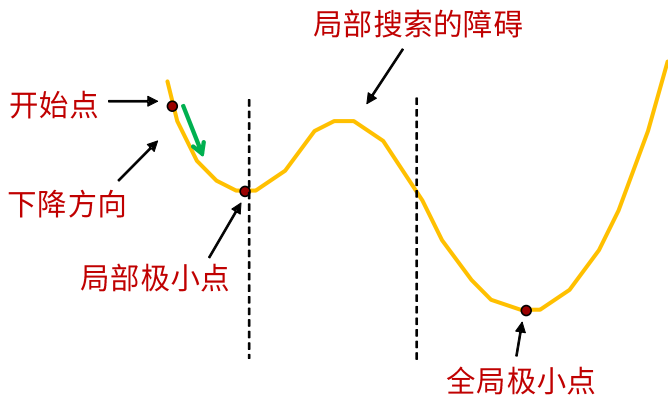
- 近年来，以深度强化学习为代表的新型优化方法日益流行，值得大家关注！
- 但也不可否认，很多研究中都将强化学习与模拟退火、遗传算法等结合进行优化！

本章主要讲解传统优化方法中的线性规划算法，以及现代优化算法中的模拟退火、遗传算法、粒子群算法，不涉及强化学习算法！



局部优化及其局限性

- 局部优化算法主要解决凸问题或单峰问题，采用确定性搜索策略，比如单纯形法、梯度下降法、爬山法、贪心算法等
- 局部优化的含义是在状态转移过程中，只接受更好的状态，拒绝恶化的状态





全局优化

- 全局性优化算法主要用于求解非凸问题或多峰问题，通常使用概率性搜索策略，即状态转移规则
- 实际的全局性优化问题通常没有解析表达式或者解析表达式非常复杂难以进行理论分析
- 全局优化算法的基本思想是在可行域中从给定的一个或多个随机初始解出发进行搜索，利用适当的状态转移规则和概率性的状态接受规则搜索新的优化解
- 局部优化算法通常能够快速收敛到局部最优解，全局优化算法通过概率搜索可以获得**概率意义上的全局最优解**，但是其局部优化搜索能力较低



数值优化与组合优化

数值优化与组合优化

最优化问题一般由目标函数和约束条件两部分构成：

$$\text{Minimize } f(X) = f(x_1, x_2, \dots, x_n)$$

$$\text{subject to } X = (x_1, x_2, \dots, x_n) \in S \subset X$$

当 $X = R^n$ ，即定义域为 n 元实空间时，该最优化问题为数值优化问题；当 X 为离散变量时，问题为组合优化问题。

数值优化问题 —— Shubert 函数

$$\begin{aligned} f(x, y) = & \left\{ \sum_{i=1}^5 i \cos[(i+1)x + 1] \right\} \left\{ \sum_{i=1}^5 i \cos[(i+1)y + 1] \right\} \\ & + 0.5[(x + 1.42513)^2 + (y + 0.80032)^2] \end{aligned}$$

自变量取值范围 $-10 < x, y < 10$ ，此函数共有 760 个局部极小点，只有一个为全局最优解。



数值优化实例

数值优化问题 —— Camel 函数

$$f(x, y) = (4 - 2.1x^2 + \frac{x^4}{3})x^2 + xy + (-4 + 4y^2)y^2$$

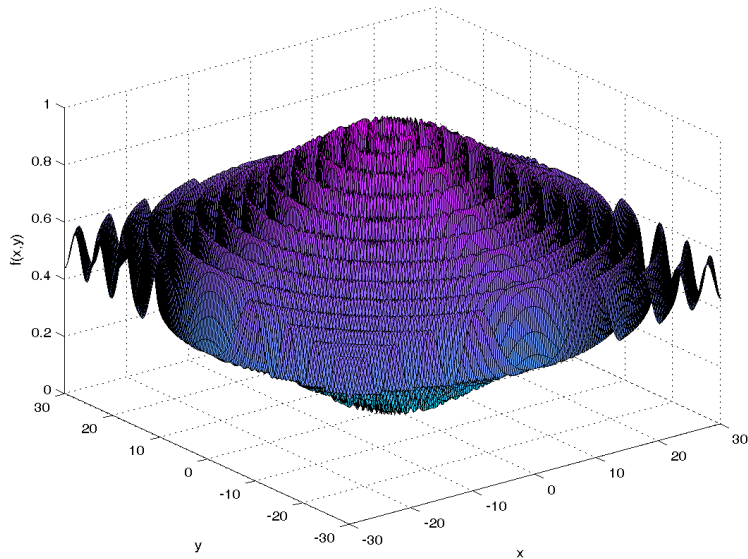
自变量取值范围 $-100 < x, y < 100$ ，此函数共有 6 个局部极小点，有 2 个全局最优解点。

数值优化问题 —— Shaffer's F_6 函数

$$f(x, y) = 0.5 - \frac{\sin^2 \sqrt{x^2 + y^2} - 0.5}{(1 + 0.001(x^2 + y^2))^2}$$

自变量取值范围 $-100 < x, y < 100$ ，此函数有无穷个局部极大点，只有 (0,0) 为全局最优解点。

Shaffer's F_6 函数图像





组合优化实例

组合优化常见问题有旅行商问题 (TSP)、作业调度问题 (JSP)、背包问题等等!

TSP 问题复杂度:

- TSP 问题的复杂度为 $(n-1)!/2$
- 5 城市 TSP 的可能线路有 12 条
- 10 城市 TSP 的可能线路为 181440 条
- 20 城市 TSP 的可能线路数为 6.0823×10^{16} 条, 假设一台 PC 每秒处理 1 亿条线路, 则需 19.3 年才可能计算完
- 100 城市 TSP 的可能线路组合为 4.6663×10^{155} 条, 而目前宇宙中总原子数量只有 10^{80} , 据说宇宙自大爆炸至今总共只进行了 10^{120} 次操作 (每原子每秒约 260 万亿亿次操作)
- JSP 问题往往比 TSP 问题更复杂



提纲

1. 现代优化算法简介

2. 线性规划算法

2.1 线性规划算法简介

2.2 规划算法的专业相关应用及其求解器

3. 模拟退火算法

3.1 模拟退火算法求解旅行商问题

3.2 模拟退火算法求解套裁问题

3.3 模拟退火的专业相关应用 (以 Placement 为例)

4. 遗传算法

4.1 遗传算法数值优化

4.2 遗传算法组合优化之旅行商问题

4.3 遗传算法的专业相关应用

5. 粒子群算法

5.1 粒子群算法用于数值优化

5.2 粒子群算法的专业相关应用



线性规划问题

线性规划问题是一类决策最优化问题，该类问题可建模为求解多变量线性函数的最优解问题，且这些变量需要满足一些线性等式或线性不等式约束

假设你得到一笔 1 亿元的现金用于投资，常见的投资形式为股票、债券、现金，这三种投资形式的预期年化收益率分别为 10%、7%、3%。考虑到资金安全，要求股票上的投资不能超过债券投资的 $\frac{1}{3}$ ，现金投资至少相当于股票和债券投资总额的 25%。请问应该如何组合投资才能使你的收益最大化？



投资问题的建模

假设 x , y , z 分别表示投资于股票、债券、现金的金额 (单位为百万元)

$$\begin{array}{ll}\max & 0.10x + 0.07y + 0.03z \\ \text{s.t.} & x + y + z = 100 \\ & x \leq \frac{1}{3}y \\ & z \geq 0.25(x + y) \\ & x \geq 0, y \geq 0, z \geq 0\end{array}$$



线性规划简史

- 1939 年，苏联学者 Kantorovich 为前苏联政府解决优化问题时提出了极值问题，被认为是线性规划的雏形
- 同期，美国的线性规划获得了飞快的发展。在应用层面，1941 年，Hitchcock 提出运输问题；1945 年，Stigler 提出了营养问题；1945 年，Koopmans 提出了经济问题
- 在线性规划的理论基础方面，“线性规划之父” G.B.Dantzig 在 1947 年担任美国空军审计官的数学顾问时，提出了“在一组线性方程或不等式约束下，求某一线性形式极小值问题的数学模型”，即“线性规划” (linear programming)
- 1947 年夏天，Dantzig 还提出了线性规划的单纯形算法。这个算法在后来被评为 20 世纪最伟大的算法之一



线性规划问题的求解方法发展

- 单纯形法 (Simplex Method) 是解决线性规划的经典方法
- 但是 1971 年, Klee 和 Minty 两位学者构造出一个例子, 该例子下单纯形法需要访问指数数量级别的顶点, 即在最坏情况下, 单纯形法是一个指数时间算法
- 1979 年, L.G. Khachiyan 发明了椭球算法 (Ellipsoid Method) 这是第一个解决线性规划问题的多项式时间算法。但是, 这个算法被证明不切实际, 仅具有理论价值
- 1984 年, N. Karmarkar 发明了内点算法 (interior point method), 这是线性规划第一个实际可用的多项式时间算法



线性规划模型

线性规划是一类经典的优化模型，模型包括目标函数、决策变量和约束条件三部分。例如：

$$\max \quad x_1 - 2x_2 \quad (1)$$

$$\text{s.t.} \quad x_1 + x_2 \leq 40 \quad (2)$$

$$2x_1 + x_2 \leq 60 \quad (3)$$

$$x_1, x_2 \geq 0 \quad (4)$$

上述的典型线性规划模型中，式 (1) 是目标函数， x_1, x_2 是决策变量，式 (2)-(4) 为约束条件，其中式 (2) 和式 (3) 为线性约束，式 (4) 为非负约束。对线性规划模型而言，目标函数和约束条件都是线性函数。线性函数可以理解为变量的最高阶数为 1，即不会出现 x_1x_2 ， x_1^2 等阶数超过 1 的情况



线性规划的模型假设

- 比例假设：目标函数中的系数是固定的常数，例如生活中类似“2 元钱 1 个，3 元钱 2 个”这种促销手段，用线性规划是无法描述的
- 非负假设：变量必须取非负值，且不要求必须是非负整数
- 确定性假设：目标函数中的系数、约束中的常数和变量系数，都是已知且不变的常数。但在生产实践中，这些参数可能在某一范围内变化，此时需要考虑采用鲁棒优化 (Robust Optimization)、随机优化 (Stochastic Optimization) 等方法求解



线性规划的标准型

在标准型中,已知 n 个实数 $c_1, c_2, \dots, c_n$; m 个实数 $b_1, b_2, \dots, b_m$ 以及 mn 个实数 a_{ij} , 其中 $i = 1, 2, \dots, m$, $j = 1, 2, \dots, n$ 。
线性规划需要找出 n 个实数 $x_1, x_2, \dots, x_n$, 满足

$$\begin{aligned} \max \quad & \sum_{j=1}^n c_j x_j \\ \text{s.t.} \quad & \sum_{j=1}^n a_{ij} x_j \leq b_i, \quad i = 1, 2, \dots, m \\ & x_j \geq 0, \quad j = 1, 2, \dots, n \end{aligned}$$



整数线性规划

- 无论是单纯形法还是 Karmarkar 算法，都不允许限定变量值只取整数值的整数线性规划问题
- 如果线性规划中的变量必须是整数，则说该线性规划问题是一个整数线性规划 (integer linear programming, ILP) 问题
- 除极个别问题外，ILP 问题求解非常困难，没有多项式时间算法，无法用单纯形法和 Karmarkar 算法求解



分数背包问题的线性规划建模

背包问题

给定一个承重为 W 的背包和 n 个重量为 $w_1, w_2, \dots, w_n$, 价值为 $v_1, v_2, \dots, v_n$ 的物品, 求这些物品中最有价值的一个子集, 且子集中所有物品都能够装到背包中。

分数背包问题的线性规划模型

$$\begin{array}{ll}\max & \sum_{j=1}^n v_j x_j \\ \text{s.t.} & \sum_{j=1}^n w_j x_j \leq W \\ & 0 \leq x_j \leq 1, \quad j = 1, 2, \dots, n\end{array}$$



0-1 背包问题的线性规划建模

0-1 背包问题的线性规划模型

$$\begin{array}{ll}\max & \sum_{j=1}^n v_j x_j \\ \text{s.t.} & \sum_{j=1}^n w_j x_j \leq W \\ & x_j \in \{0, 1\}, \quad j = 1, 2, \dots, n\end{array}$$

分数背包问题和 0-1 背包问题的线性规划模型唯一区别在于 x_j 是否可以取分数，而且貌似 0-1 背包问题更简单一些，但 0-1 背包问题的线性规划模型是 ILP 问题，求解难度要高得多



非线性规划问题

- 若规划问题的目标函数或约束条件中包含非线性函数，则称为非线性规划
- 非线性规划的最优解（若存在）可能在其可行区域的任一点达到
- 目前非线性规划还没有适合各种问题的一般解法，各种方法都有其特定的适用范围



非线性规划问题的标准型

min	$F(X)$	(目标函数)
s.t.	$AX \leq b$	(线性不等式约束)
	$Aeq \cdot X = beq$	(线性等式约束)
	$C(X) \leq 0$	(非线性不等式约束)
	$Ceq(X) = 0$	(非线性等式约束)
	$VLB \leq X \leq VUB$	(有界约束)



二次规划

二次规划 (QP, Quadratic Programming) 定义：目标函数为二次函数，约束条件为线性约束，属于最简单的一种非线性规划。

$$\begin{array}{ll} \min & f(X) = \frac{1}{2}x^T Qx + c^T x \\ \text{s.t.} & AX \leq b \end{array}$$

当 Q 正定时，用椭圆法可在多项式时间内解二次规划问题。当 Q 非正定时，二次规划问题是 NP 难的 (NP-Hard)。即使 Q 只存在一个负特征值时，二次规划问题也是 NP 难的



线性规划的简单实例

求解两变量线性规划问题 $3x + 5y$ 的最大值。约束为：

$$x + y \leq 4$$

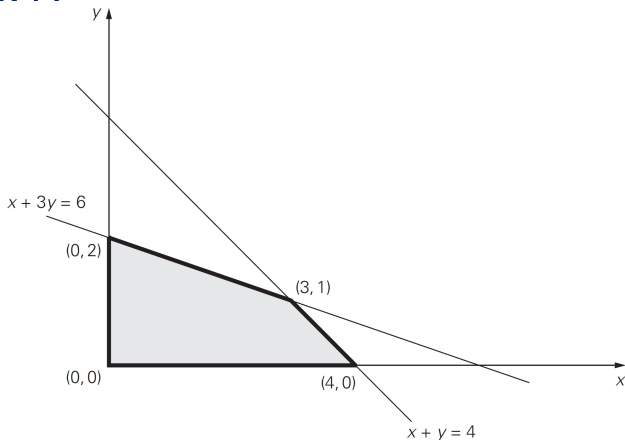
$$x + 3y \leq 6$$

$$x \geq 0, y \geq 0$$

- 该问题的可行解 (feasible solution) 是满足该问题所有约束的任意点 (x, y)
- 该问题的可行区域 (feasible region) 是所有可行点的集合



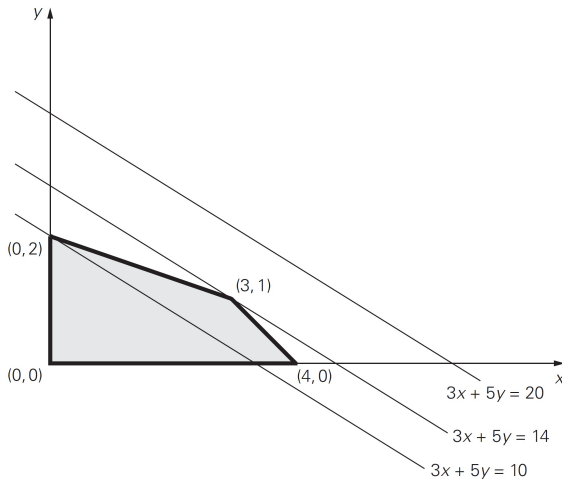
可行区域图示



- 上图中阴影部分即为上例的可行区域，它是由两个不等式约束以及非负约束围成的凸多边形区域
- 要求在可行区域内找到一个最优解 (optimal solution), 使得目标函数 (objective function) $z = 3x + 5y$ 取最大值



线性规划的几何解法示例



$z = 3x + 5y$ 定义了一簇斜率相同的直线，称为该目标函数的水平线 (level line)。将某条直线 $z = 3x + 5y$ 由右上角向左下角移动，与可行区域的第一个交点 (3,1) 即为问题的优化解



极点定理

- 不是所有的线性规划问题都能在可行区域的顶点上找到最优解，一个线性规划问题的可行区域可能为空，也可能无界，此时无解
- 前述两变量的线性规划问题可以推广到多维，此时可行区域可能不是简单的凸多边形，而变成高维的凸多面体，该多面体具有有限数量的顶点，称为极点

极点定理

可行区域非空的任意线性规划问题有最优解，而且，最优解总是在其可行区域的某极点上出现。



几何解法的推广

- 由极点定理，在可行区域有界的情况下，总能找到问题的优化解
- 可以为可行区域的每个极点计算目标函数值，从中挑选最优值对应的极点作为优化解
- 有算法可以找到全部极点，但随着问题规模的增长，极点的数量呈指数增长，为求解带来困难
- 单纯形法有很高的概率可以在只检测可行区域的一小部分极点的情况下找到最优解

IEEE 数据库中的 ILP 算法 (应用最广) 相关研究

Showing 1-25 of 73 results for ("Abstract":programming) AND ("Abstract":ILP) DAC OR DATE OR ICCAD OR TCAD OR TC OR TPDS

Filters Applied: 2020 - 2025

☐ Journals (39)

☐ Conferences (31)

☐ Early Access Articles (3)



Need access to IEEE

Xplore

for your organization?

CONTACT IEEE TO SUBSCRIBE

Show

☒ All Results

☐ Open Access Only

Year

☒ Range

☐ Single Year

2020

2024

Clear

Apply



Author

Sign In to Save Your Search

Get notified when new research is published matching your search criteria.

* Email Address

* Password

Sign In

[Forgot Password?](#) | [Create Account](#)

☐ Select All on Page

Sort By

Relevance

☐ **Enhanced and Efficient Guiding Template Design for Lamellar DSA With Graph Monomorphism**

Yi-Sian Ciou; An-Jie Shih; Shao-Yun Fang; Yi-Yu Liu

IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

Year: 2023 | Volume: 42, Issue: 11 | Journal Article | Publisher: IEEE

Abstract

[HTML](#)



☐ **Enabling ILP-Based DSE for Multi Granularity, Unified Domain Platforms With DmTSAR-ILP**

Bruno Moraes; Qucheng Jiang; Gunar Schirner

IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

Year: 2024 | Early Access Article | Publisher: IEEE





开源规划算法求解器

- SCIP: 用于混合整数 (非线性) 规划、约束整数规划求解
- GLPK: GNU 下的一个项目, 用于建立大规模线性规划 LP 和混合型整数规划 MIP 问题, 并对模型进行最优化求解
- Lpsolve: sourceforge 下的一个开源项目, 混合整数线性规划求解器, 可以求解纯线性、(混合) 整数/二值、半连续和特殊有序集模型
- yalmip: 不仅自己包含基本的线性规划求解算法, 比如 linprog (线性规划)、bintprog (二值线性规划)、bnb (分支界定算法) 等, 他还提供了对 cplex、GLPK、lpsolve 等求解工具包更高层次的包装
- CBC 和 SYMPHONY: COIN-OR 旗下的两大求解器
- 国内开发的求解器: LEAVES、CMIP



求解器性能对比

- 开源的求解器中，以德国的 SCIP 和美国的 Coin-OR 为线性和整数规划优秀代表，二次规划里 Sedumi, SDPT3 和 DSDP 比较优秀
- 商业求解器中，美国 IBM 的 CPLEX, Gurobi, 英国的 Xpress, 三家的线性和整数规划求解器速度和稳定性最好，丹麦的 MOSEK 在二次规划和锥优化优势明显
- 最好的开源求解器 SCIP 在整数规划上的表现，在中小型问题上跟 Gurobi 和 CPLEX 有七倍左右差距，大问题上差距可能更明显
- 如下网址有开源求解器的列表，供参考：<https://zrjer.github.io/2020/02/26/OpenSourceSolver/>



提纲

1. 现代优化算法简介

2. 线性规划算法

2.1 线性规划算法简介

2.2 规划算法的专业相关应用及其求解器

3. 模拟退火算法

3.1 模拟退火算法求解旅行商问题

3.2 模拟退火算法求解套裁问题

3.3 模拟退火的专业相关应用 (以 Placement 为例)

4. 遗传算法

4.1 遗传算法数值优化

4.2 遗传算法组合优化之旅行商问题

4.3 遗传算法的专业相关应用

5. 粒子群算法

5.1 粒子群算法用于数值优化

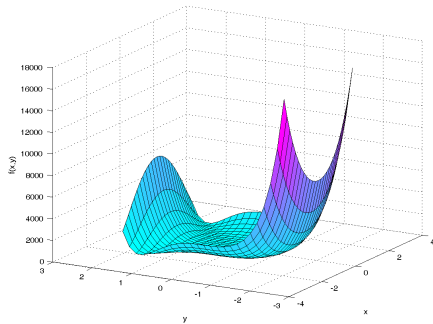
5.2 粒子群算法的专业相关应用



模拟退火的思想

- 模拟退火 (simulated annealing) 算法是局部搜索算法的扩展，但它以一定的概率选择邻域中性能更差的解
- 模拟退火算法最早的思想由 Metropolis 在 1953 年提出，Kirkpatrick 在 1982 年成功地应用于组合最优化问题 (Kirkpatrick, etc. 1983. Optimization by Simulated Annealing. Science, vol 220, No. 4598, pp 671-680.)

模拟退火与爬山法的不同在于它可能以一定的概率接受劣解，从而为跳出局部最优创造了机会





波尔兹曼分布

波尔兹曼分布

对于金属内部，在给定温度 T ，其原子停留在状态 r 的概率满足：

$$P\{\bar{E} = E(r)\} = \frac{1}{Z_T} \exp\left(-\frac{E(r)}{k_B T}\right)$$

其中 $E(r)$ 表示状态 r 的能量， $k_B > 0$ 为波尔兹曼常数 (对给定波尔兹曼分布， $k_B T$ 是常数)， $Z_T = \sum_{r \in D} \exp\left(-\frac{E(r)}{k_B T}\right)$ 是一个标准化因子， D 可认为是状态空间

- 金属加热时，原子热运动不断增强，运动越来越无序，系统熵增加；金属冷却时，原子运动变得有序，熵减小
- 对 $E_1 < E_2$ 和同一温度 T ， $P\{\bar{E} = E_1\} > P\{\bar{E} = E_2\}$ (递减函数)，即原子停留在低能量状态的概率比高能量状态大
- 温度趋向于 0 时，原子停留在最低能量状态的概率趋近于 1



Metropolis 准则

Metropolis 准则

对于原子在原始状态 i 和新状态 j 之间的跃迁，需要考虑状态能量的影响，如果 $E_j < E_i$ ，则接受新状态；如果 $E_j > E_i$ ，则需要考虑热运动的影响，是否接受新状态取决于下式：

$$r = \exp\left(\frac{E_i - E_j}{k_B T}\right)$$

r 是一个小于 1 的数，给定一个随机数 $s \in (0, 1)$ ，如果 $s < r$ ，则接受新状态，否则放弃



温度对 Metropolis 准则的影响

$$r = \exp\left(\frac{E_i - E_j}{k_B T}\right)$$

Metropolis 准则使得在较高温度下可接受与当前状态能差较大的新的高能量状态，在低温时只能接受能差较小的新高能状态：

- 1) $E_j < E_i$, $r > 1$, 必定接受
- 2) $E_j > E_i$, $r < 1$, E_j 越大 r 越小, 越不易接受, 此时高 T 值会使 r 增大, 所以高温值有利于接受高能量状态



优化与退火过程的关系

优化	退火
优化问题	金属物体
解	状态
最优解	能量最低状态
代价函数	能量

模拟退火仿真了金属冷却的过程，将寻找最优解的过程映射为向低温状态的转移过程！



Metropolis 过程

对于最小优化问题：

- 1) 给定初始温度 T_0 和初始点 x_0 ，计算该点的函数值 $f(x_0)$
- 2) 产生随机扰动 Δx ，得到新点 $x' = x + \Delta x$ ，计算新点函数值 $f(x')$ 和差值 $\Delta f = f(x') - f(x)$
- 3) 若 $\Delta f \leq 0$ ，则接受新点作为下一次模拟退火的初始点
- 4) 若 $\Delta f > 0$ ，计算新点接受概率 $P(\Delta f) = \exp(-\Delta f/kT)$ ，并产生随机数 $s \in (0, 1)$ ，若 $s \leq P(\Delta f)$ ，则接受新点，否则仍留用原有点



模拟退火算法

SIMULATED-ANNEALING()

- 1: 设定初温 $t = t_0$ ，随机产生初始状态 $s = s_0$ ，令 $k = 0$
- 2: **repeat**
- 3: **repeat**
- 4: $s_j = \text{generate}(s)$ ▷ 产生新状态 s_j
- 5: **if** $\min\{1, \exp(-(C_{s_j} - C_s)/t_k)\} \geq \text{random}[0, 1]$ **then**
- 6: $s = s_j$
- 7: **until** 抽样稳定准则满足
- 8: $t_{k+1} = \text{update}(t_k)$ ▷ 降温
- 9: $k = k + 1$
- 10: **until** 算法终止准则满足
- 11: **return** 算法搜索结果

模拟退火算法可简单归纳为三函数两准则



状态产生函数和状态接受函数

- 状态产生函数
 - 尽可能使产生的候选解遍布整个状态空间
 - 可以采用 n -opt 规则生成新状态
 - 状态产生可服从一定概率分布
- 状态接受函数
 - 固定温度下，低目标值候选解优于高目标值候选解被接受
 - 随温度降低，接受劣化解 (高目标函数值) 概率逐步降低
 - 温度趋近于 0，仅接受目标函数值下降的解
 - 状态接受函数是模拟退火算法的核心



初温设定和温度更新函数

- 初温设定
 - 初温越高，得到优化解的概率越大
 - 可均匀抽样一组状态，以状态目标值的方差为初温
 - 随机产生一组状态，求目标差值的最大值 $|\Delta_{max}|$ ，以之为基础，令 $t_0 = -|\Delta_{max}|/\ln p_\tau$ ， p_τ 为初始接受概率
 - 依据经验值确定
- 温度更新函数
 - $t_k = \alpha/\log(k + k_0)$
 - 快速更新: $t_k = \beta/(1 + k)$
 - 指数退温: $t_{k+1} = \lambda t_k$, $\lambda \in (0, 1)$



循环终止准则

- 内循环终止准则
 - 也称 Metropolis 抽样稳定准则
 - 检验目标函数均值是否稳定
 - 连续若干步目标值变化较小
 - 按一定步数抽样
- 外循环终止准则
 - 即算法终止准则
 - 设置终止温度阈值
 - 设置外循环迭代步数
 - 最优值连续若干步不变
 - 检验系统熵是否稳定



模拟退火求解旅行商问题思路

求解思路：将旅行商问题转化为求解城市的一个最优排列问题：

- 1) 产生一条新的遍历路径 $P(i+1)$ ，计算路径 $P(i+1)$ 的代价 $L(P(i+1))$
- 2) 若 $L(P(i+1)) < L(P(i))$ ，则接受 $P(i+1)$ 为新的路径，否则以概率接受 $P(i+1)$ ，迭代，然后降温
- 3) 重复步骤 1，2 直到满足退出条件

产生新的遍历路径的方法有很多，下面列举 3 种：

- 1) 随机选择 2 个节点，交换路径中的这 2 个节点的顺序
- 2) 随机选择 2 个节点，将路径中这 2 个节点间的节点顺序逆转
- 3) 随机选择 3 个节点 m, n, k ，然后将 m 与 n 间的节点移位到节点 k 后面

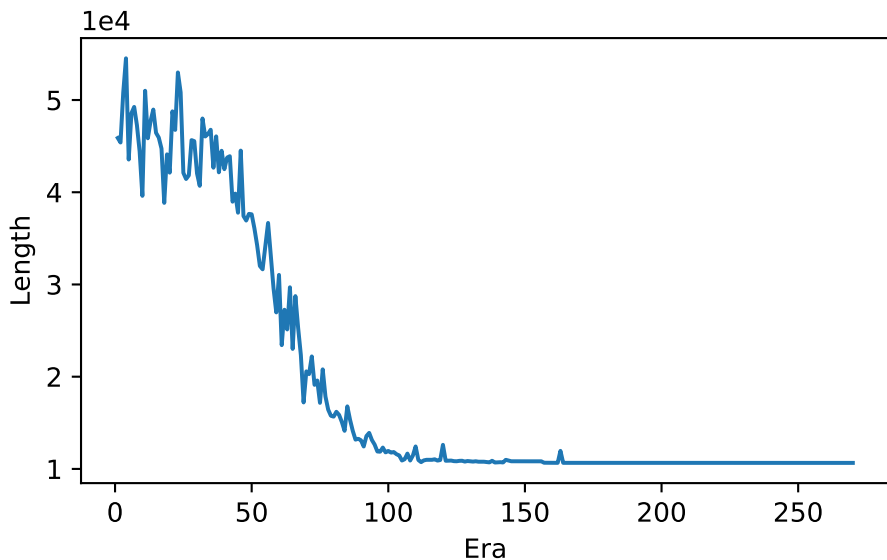


模拟退火算法求解 TSP 问题示例 —— 48 城市 TSP

```
main.cpp tsp.h x
SA_TSP (Global Scope) SA()
157 while (t > Tf)
158 {
159     for (i = 0; i < IN_loop; i++)
160     {
161         loop_counter++;
162
163         getnextpath(curpath, &newpath);
164         delta = newpath.length - curpath.length;
165         if (delta < 0)
166         {
167             copy_path(newpath, &curpath);
168         }
169         else
170         {
171             p = rand()*1.0 / RAND_MAX;
172             if (exp(-delta / t) > p)
173                 copy_path(newpath, &curpath);
174         }
175     }
```

```
C:\Windows\system32\cmd.exe
C:\SA_TSP>SA_TSP.exe
average: 10663.5
best: 10598
worst: 10801
loop counter: 2700000
C:\SA_TSP>
```

模拟退火求解 48 城市 TSP 的目标值演化

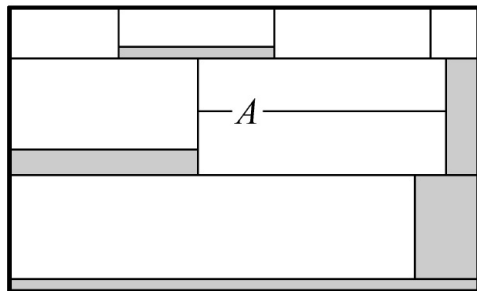
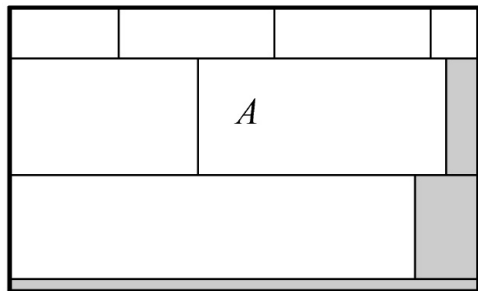




套裁问题

套裁问题

套裁问题，又称下料问题 (cutting stock problems)，存在于诸如玻璃、钢板、木材、纸张和制衣等的裁剪问题中。它们共同的特点是原料的尺寸大于需求的尺寸，而需求的品种尺寸可以不相同，最终的目标是在满足需求的前提下，使得边角废料最小。





套裁问题的形式化

给定初始矩形原料 $W_0 \times H_0$, 要从中裁出 n 个候选零件 $R = \{R_1, R_2, \dots, R_n\}$, $I = \{1, 2, \dots, n\}$ 表示矩形零件的编号, 目标函数 S 可设为包络矩形的面积, (x_l^i, y_l^i) 和 (x_u^i, y_u^i) 分别表示第 i 个部件的左下角和右上角坐标, w_i 和 h_i 分别表示零件 i 的宽和高, l_{ij} 和 b_{ij} 分别表示零件 i 和 j 的位置关系, 若零件 i 置于零件 j 的左侧, $l_{ij} = 1$, 否则为 0; 若零件 i 置于零件 j 的下方, $b_{ij} = 1$, 否则为 0。则套裁问题的形式化描述如下:

$$\text{Min } S \quad (5)$$

$$x_u^i \leq W_0, y_u^i \leq H_0, i \in I \quad (6)$$

$$x_u^i = x_l^i + w_i, y_u^i = y_l^i + h_i, i \in I \quad (7)$$

$$x_u^i \leq x_l^j + W_0(1 - l_{ij}), i \neq j \in I \quad (8)$$

$$y_u^i \leq y_l^j + H_0(1 - b_{ij}), i \neq j \in I \quad (9)$$

$$l_{ij} + l_{ji} + b_{ij} + b_{ji} \geq 1, i, j \in I, i < j \quad (10)$$

$$S, x_u^i, x_l^i, y_u^i, y_l^i \geq 0, i \in I \quad (11)$$

$$l_{ij}, b_{ij} = \{0, 1\}, i, j \in I \quad (12)$$

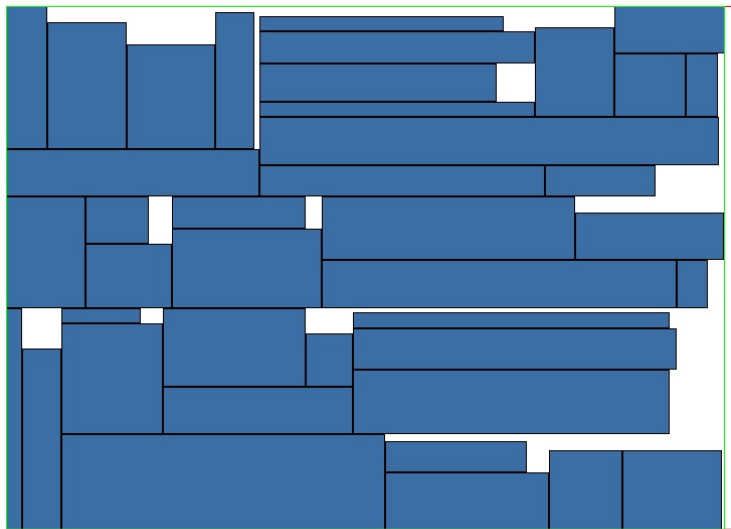


套裁问题的模拟退火算法设置

- 用小矩形的排列作为问题的编码,新解的生成采用 2-opt 的方法,即任意交换两个元素的位置,比如初始解是 $\{1,2,3,4\}$, 则进行一次 2-opt 操作之后可能变成 $\{1,4,3,2\}$, 即交换了 2、4 两个元素的位置
- 降温函数采用 Lundy 和 Mees 的 $t_{k+1} = t_k(1 + \beta t_k)^{-1}$, 其中 $\beta = (t_0 - t_f)/(M t_0 t_f)$, t_f 为给定值, M 为外循环总次数
- 内循环终止准则: 设要切割产品数为 n , 则内循环次数 $n \sim n^2$ 次
- 外循环终止准则: 外循环进行 M 次后终止



套裁问题运行实例



Initial Rectangle Width460.00 External Rectangle Width454.00 ScrapRate:9.56%
Initial Rectangle Height330.00 External Rectangle Height330.00



模拟退火在微电子领域的应用

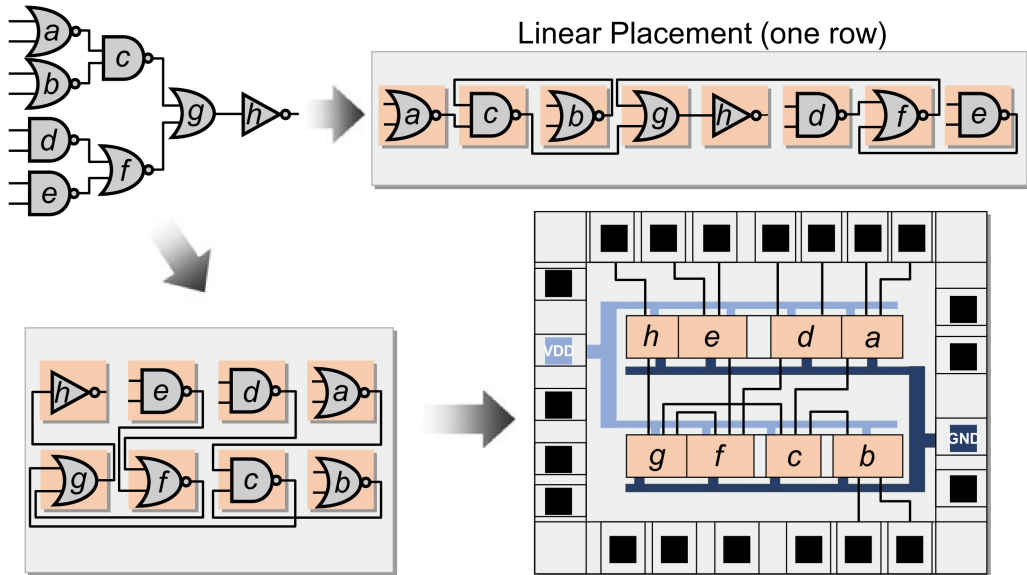
- Placement
- Routing
- Array logic minimization
- Layout

作为早期著名的可重构处理器架构，IMEC 的 ADRES 处理器在其编译器 DRESC 中即采用 SA 作为其调度算法的核心 (FPT'02)!

上述问题也可以采用其它优化算法，比如遗传算法，进行求解!

近期 Google 采用增强学习算法进行 Placement，取得了很好的效果：arXiv:2004.10746, Chip Placement with Deep Reinforcement Learning.

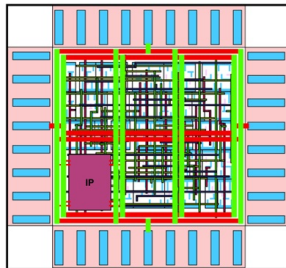
标准单元 (Standard Cell) 的 Placement 问题



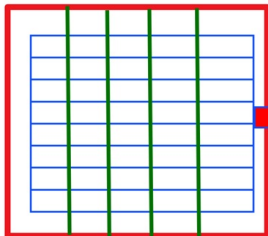


芯片版图及 Placement 问题示意

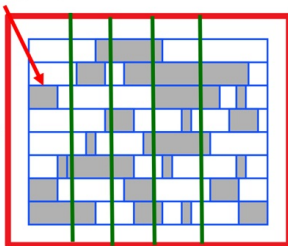
在基于标准单元的设计流程中，
Placement指将标准单元按一定的
优化目标放置到Core Area中，标
准单元等高不等宽，按行放置



Standard Cell



Design Before Placement



Design After Placement

Sites in the LEF File

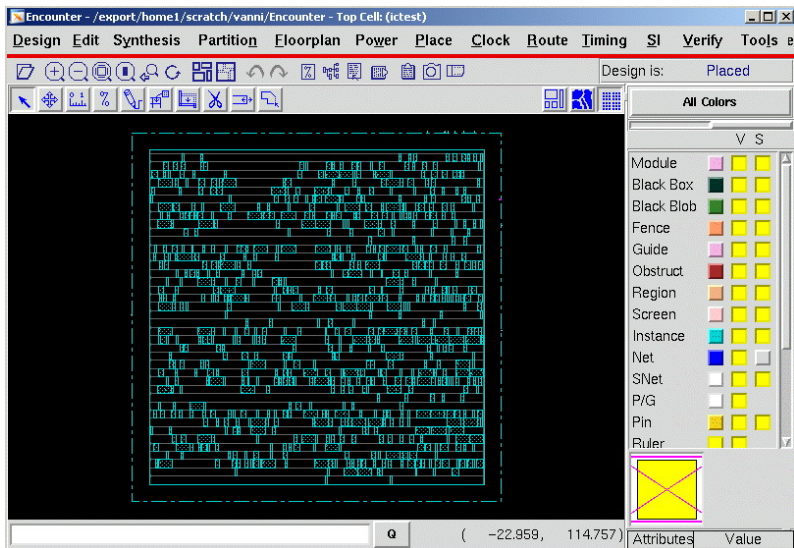
```
SITE coreSite
  SYMMETRY y ;
  CLASS core ;
  SIZE 0.660 BY 5.040 ;
END coreSite

SITE pad
  SYMMETRY x y r90 ;
  CLASS pad ;
  SIZE 0.100 BY 235.000 ;
END pad

SITE corner
  SYMMETRY x y r90 ;
  CLASS pad ;
  SIZE 235.000 BY 235.000 ;
END corner
```



EDA 软件中的 Placement 效果



图片来源: <https://limsk.ece.gatech.edu/course/ece6133/lab/encounter.html>



Placement 应用模拟退火需注意的问题

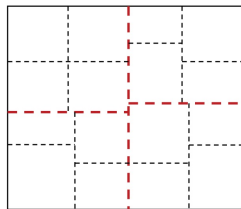
- 为每个门找到合适的放置位置，使得放置后电路具有良好的可布线性、低延迟、小面积
- Placement 还需注意 I/O 的摆放、芯片矩形形状限制、曼哈顿布线算法限制
- 考虑上述限制，Placement 问题应该是一个多目标优化的问题，其代价函数可设置为：

$$Cost = c_1 area + c_2 delay + c_3 power + c_4 crosstalk$$

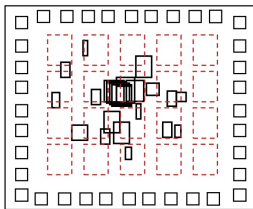
- 代价函数的各项系数应根据芯片实际需求决定

资料来源: Sherwani, “Algorithms for VLSI Physical Design Automation”, Kluwer Academic Publishers, 1999.

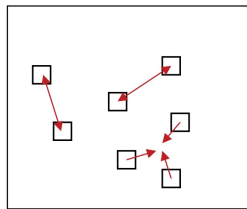
从 Routing 质量衡量 Placement 效果



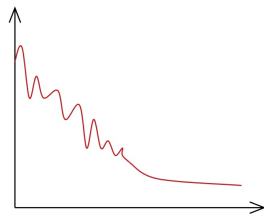
Min-Cut
Partitioning



Quadratic
Placement



Force-Directed
Placement



Simulated
Annealing



用于 Placement 的模拟退火算法主框架

Simulated Annealing Algorithm for Placement

Input: set of all cells V

Output: placement P

```
1   $T = T_0$                                 // set initial temperature
2   $P = PLACE(V)$                             // arbitrary initial placement
3  while ( $T > T_{min}$ )
4    while ( $!STOP()$ )                      // not yet in equilibrium at  $T$ 
5       $new\_P = PERTURB(P)$ 
6       $\Delta cost = COST(new\_P) - COST(P)$ 
7      if ( $\Delta cost < 0$ )                  // cost improvement
8         $P = new\_P$                         // accept new placement
9      else                                // no cost improvement
10          $r = RANDOM(0, 1)$                 // random number  $[0, 1)$ 
11         if ( $r < e^{-\Delta cost/T}$ )        // probabilistically accept
12            $P = new\_P$ 
13   $T = \alpha \cdot T$                       // reduce  $T$ ,  $0 < \alpha < 1$ 
```

基于模拟退火的 TimberWolf Placement 算法

- TimberWolf 由 UC Berkeley 的 Sechen 开发，后被商业化
- TimberWolf 的主要步骤为：
 1. Place standard cells with minimum total wirelength using simulated annealing
 2. Globally route the placement by introducing routing channels, when necessary. Recompute and minimize the total wirelength
 3. Locally optimize the placement with the goal of minimizing channel height

[1] Sechen C, Sangiovanni-Vincentelli A. The TimberWolf placement and routing package[J]. IEEE Journal of Solid-State Circuits, 1985, 20(2): 510-522.



TimberWolf 算法中的 Perturb 操作

Perturb generates a new placement from an existing placement using one of the following actions:

1. MOVE: Shift a cell to a new position (another row)
2. SWAP: Exchange two cells
3. MIRROR: Reflect the cell orientation around the y-axis, used only if MOVE and SWAP are infeasible



模拟退火算法的其他应用

- 在论文 [1] 中，提出了一种启发式的 EMS 模调度算法，只花费很少的时间（与模拟退火相比）即可达到模拟退火算法 98% 的调度性能
- 论文 [2] 中，面向 DNN 加速器提出了基于增强学习的资源分配算法，优于贝叶斯优化、遗传算法、模拟退火算法，且算法运行速度快 4.7 24 倍
- 论文 [3] 中，提出了一种可以减少配置擦洗时间的改进型模拟退火布局算法，以增强 FPGA 系统的可靠性

[1] Edge-centric modulo scheduling for coarse-grained reconfigurable architectures[C]. PACT'08.

[2] ConfuciuX: Autonomous Hardware Resource Assignment for DNN Accelerators using Reinforcement Learning. MICRO'20.

[3] Scrubbing-Aware Placement for Reliable FPGA Systems[J]. IEEE Tran. on Emerging Topics in Computing, 2020.

IEEE 数据库中的模拟退火算法相关研究



☐ Journals (23)

☐ Conferences (12)

☐ Early Access Articles (2)

Search

Documents

Images (Beta)

Show

☒ All Results

☐ Subscribed Content [?](#)

☐ Open Access Only

Year [?](#)

☒ Range

☐ Single Year

2020

2024

Clear

Apply

Sign In to Save Your Search [?](#)

Get notified when new research is published matching your search criteria.

* Email Address

* Password

Sign In

[Forgot Password?](#) | [Create Account](#)

☐ Select All on Page

Sort By [Most Cited By Papers](#) ▼

☐ **Thermal TSV Optimization and Hierarchical Floorplanning for 3-D Integrated Circuits** [?](#)

Zongqing Ren; Ayed Alqahtani; Nader Bagherzadeh; Jaeho Lee

IEEE [Transactions](#) on Components, Packaging and Manufacturing Technology

Year: 2020 | Volume: 10, Issue: 4 | Journal Article | Publisher: IEEE

Cited by: [Papers \(42\)](#)

[?](#) Abstract [HTML](#) [?](#) [?](#)

Author [?](#)

Affiliation [?](#)

[?](#) Publication Title [?](#)

☐ **4.6 A 144Kb Annealing System Composed of 9×16Kb Annealing Processor Chips with Scalable Chip-to-Chip Connections for Large-Scale Combinatorial Optimization Problems** [?](#)

Takashi Takemoto; Kasho Yamamoto; Chihiro Yoshimura; Masato Hayashi; Masafumi Tada; Hiroaki Saito; Mayumi Mashimo; Masanao Yamaoka

2021 IEEE International Solid-State [Circuits](#) Conference ([ISSCC](#))



提纲

1. 现代优化算法简介

2. 线性规划算法

2.1 线性规划算法简介

2.2 规划算法的专业相关应用及其求解器

3. 模拟退火算法

3.1 模拟退火算法求解旅行商问题

3.2 模拟退火算法求解套裁问题

3.3 模拟退火的专业相关应用 (以 Placement 为例)

4. 遗传算法

4.1 遗传算法数值优化

4.2 遗传算法组合优化之旅行商问题

4.3 遗传算法的专业相关应用

5. 粒子群算法

5.1 粒子群算法用于数值优化

5.2 粒子群算法的专业相关应用



遗传算法简介

- 遗传算法 (Genetic Algorithm, GA) 的研究最早见于 20 世纪六七十年代, 主要以 Michigan 大学的 John Holland 为主
- 遗传算法的基本出发点是模拟自然界生物的复杂适应性
- 1975 年 Holland 开创性著作 “Adaptation in Natural and Artificial Systems” 的发表可视为遗传算法的正式诞生
- 遗传算法的应用领域包括但不限于: 函数优化、组合优化、生产调度、自动控制、机器人智能控制、图像处理与模式识别、人工生命、遗传程序设计、机器学习等
- 遗传算法已经成为现代优化算法中研究与应用最成熟的领域



2018 年美国限制 14 类技术出口清单

83-FR-58201: Review of Controls for Certain Emerging Tech.

- (1) Biotechnology, such as:
 - (i) Nanobiology;
 - (ii) Synthetic biology;
 - (iv) Genomic and genetic engineering;
- or
- (v) Neurotech.
- (2) Artificial intelligence (AI) and machine learning technology, such as:
 - (i) Neural networks and deep learning (e.g., brain modelling, time series prediction, classification);
 - (ii) Evolution and genetic computation (e.g., genetic algorithms, genetic programming);
 - (iii) Reinforcement learning;

<https://www.gpo.gov/fdsys/pkg/FR-2018-11-19/pdf/2018-25221.pdf>



- 遗传算法模拟了自然界生物进化的过程，优化解通过适者生存的机制得到
- 粒子群算法模仿群体（鸟群）智能，模拟鸟群觅食的有组织性及信息共享机制寻找最优解
- 蚁群算法是另一种模仿群体智能的算法，其模仿的是蚂蚁群体的觅食过程及基于信息素的信息交互机制
- 其它仿生学算法还有人工鱼算法、人工蜂群算法、萤火虫算法等



生物遗传学与进化理论基础

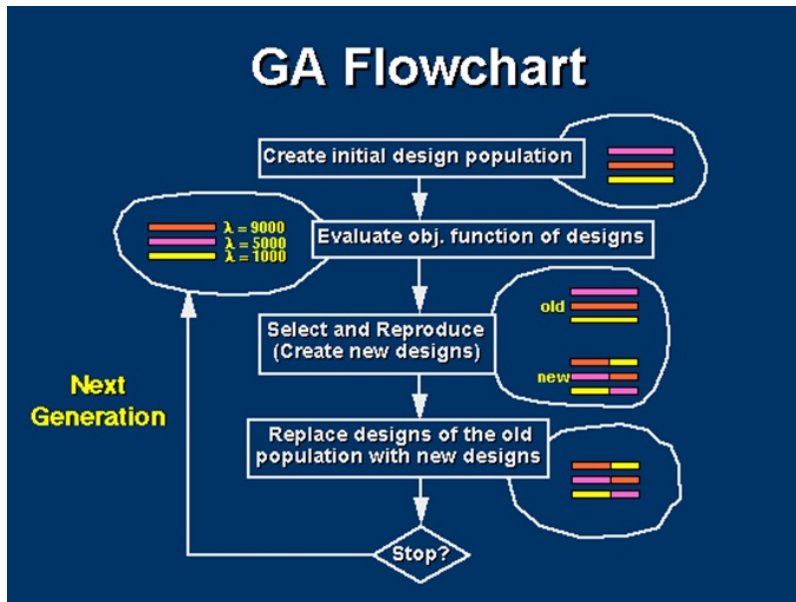
- 基因 (gene): DNA 或 RNA 中占有一定位置的基本遗传单位
- 染色体 (chromosome): 细胞中的微小丝状化合物, 包含多个基因
- 个体 (individual): 带有染色体的生命个体
- 种群 (population): 多个个体的集合称为种群
- 进化 (evolution): 生物在延续生存的过程中, 适应性和品质提高的现象称为进化, 进化是以种群的形式进行的
- 适应度 (fitness): 用来衡量个体对生存环境的适应程度
- 选择 (selection): 从种群中选择一定数量优势个体进行繁殖的过程
- 复制 (reproduction): 新细胞继承旧细胞的遗传物质的过程
- 交叉 (crossover): 两个父个体交换部分染色体形成具有新染色体的子代
- 变异 (mutation): 染色体复制时以很小概率产生的差错现象, 这增强了物种的多样性
- 编码 (coding): DNA 中遗传物质在一个长链上按一定的模式排列, 也即进行了遗传编码



遗传算法基本思想

- 遗传算法从代表问题可能解集的一个种群开始，种群中的个体实际上就是某种编码的染色体
- 染色体的遗传编码要能反映问题的解，比如函数优化的变量值、组合优化的组合方案等需要编码于染色体中
- 每个染色体可以映射到优化目标值：函数值或优化代价
- 遗传算法主要借鉴了自然进化的选择、交叉、变异这几种遗传操作，通过这些遗传操作所制造的进化压力形成优良个体，从而得到问题的优化解
- 由生物学知识，不是所有交叉得到的子代（比如退化、返祖）以及所有变异个体（比如肿瘤）都具有进化优势，不具有进化优势的个体被选择算子筛选掉
- 选择对应局部搜索，交叉和变异对应跳出局部最优的努力

遗传算法工作流程





遗传算法基本操作

适应度计算

- 直接采用目标值作为适应度
- 按比例适应度计算
- 基于排序的适应度计算

选择算子

- 轮盘赌选择
- 随机遍历抽样
- 局部选择
- 截断选择
- 锦标赛选择



遗传算法基本操作

交叉算子

- 实值重组
 - 离散重组
 - 中间重组
 - 线性重组
- 二进制交叉
 - 单点交叉
 - 多点交叉
 - 均匀交叉
 - 洗牌交叉

变异算子

- 实值变异
- 二进制变异



简单函数优化问题

求下列问题的最大值：

$$f(x) = x \sin(10\pi \cdot x) + 2.0 \quad x \in [-1, 2]$$

对上式求导，求极值点，得方程：

$$\tan(10\pi \cdot x) = -10\pi \cdot x$$

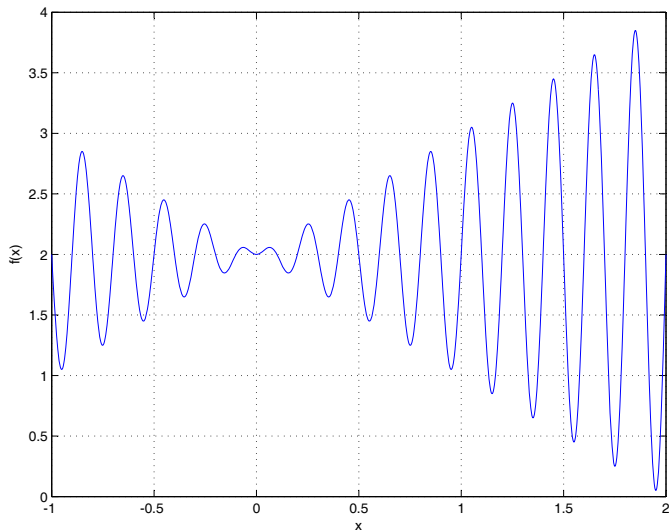
该式的解，即极值点，有无穷多个：

$$\begin{cases} x_i = \frac{2i-1}{20} + \varepsilon_i, & i = 1, 2, \dots \\ x_0 = 0 \\ x_i = \frac{2i+1}{20} + \varepsilon_i, & i = -1, -2, \dots \end{cases}$$

ε_i 是一接近于 0 的实数



数值优化的函数曲线



最大值出
现在 x_{19} ,
 $f(x_{19})$ 的值
比 3.85 稍
大!



简单数值优化的遗传算法设置

- 编码：
 - 以二进制位向量作为染色体编码，表示 x 的值
 - x 区间跨度为 3，如果求解精度要求 6 位小数的话，需将区间等分为 3×10^6 份，至少需要 22 位的位向量
 - 染色体解码方法为 $x = -1.0 + 3 \cdot \frac{x'}{2^{22} - 1}$ ，其中 x' 为二进制向量对应的整数
- 初始种群：100 个 22 位的位向量的数组或集合
- 适应度：直接用函数值作为适应度
- 遗传操作：
 - 选择算子：轮盘赌选择
 - 交叉算子：单点交叉，交叉概率 0.8
 - 变异算子：单点翻转，变异概率 0.05
- 进化代数：300 代



数值优化的遗传算法运行实例

```
SGA.CPP  GA Test (Global Scope)
124      do
125      {
126          Mate1 = Select(OldPop,SumFitness);
127          Mate2 = Select(OldPop,SumFitness);
128          Child1 = Mate1;
129          Child2 = Mate2;
130          double Rnd;
131          Rnd = rand()/static_cast<double>(RAND_MAX);
132          if(Rnd<=PCross)
133          {
134              Crossover(Mate1,Mate2,Child1,Child2);
135          }
136          Rnd = rand()/static_cast<double>(RAND_MAX);
137          if(Rnd<=PMutation)
138          {
139              if(rand()%2==0)
140              {
141                  Mutation(Child1);
142              }
143              else
```

```
C:\GATest\GATest.exe
第61代进化完成!
第71代进化完成!
第81代进化完成!
第91代进化完成!
第101代进化完成!
第111代进化完成!
第121代进化完成!
第131代进化完成!
第141代进化完成!
第151代进化完成!
第161代进化完成!
第171代进化完成!
第181代进化完成!
第191代进化完成!
第201代进化完成!
第211代进化完成!
第221代进化完成!
第231代进化完成!
第241代进化完成!
第251代进化完成!
第261代进化完成!
第271代进化完成!
第281代进化完成!
第291代进化完成!
*****
最优X值: 1.85061
最优适应度: 3.85027
最优个体产生代数: 45
*****
```




旅行商问题的遗传算法优化原理

遗传编码

旅行商问题最自然的编码方式是基于路径的编码，即对城市进行编号，编号的任意一个排列对应一种路径表示

路径的交叉操作比较困难

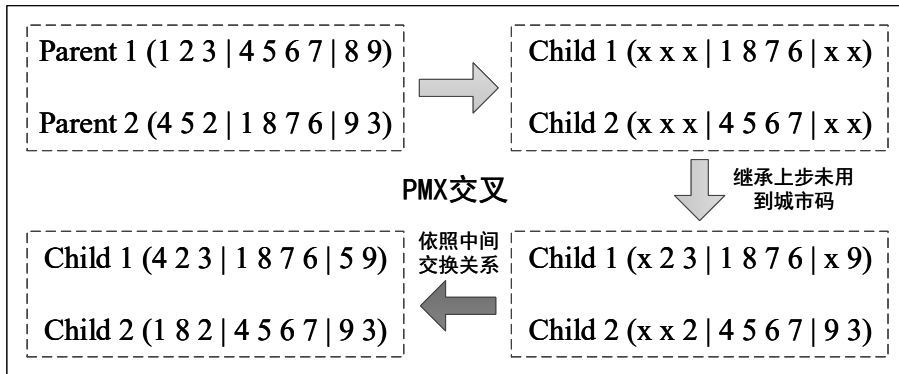
- 部分匹配交叉 PMX
- 顺序交叉 OX
- 循环交叉 CX

变异操作

交换任意两个城市位置即可！



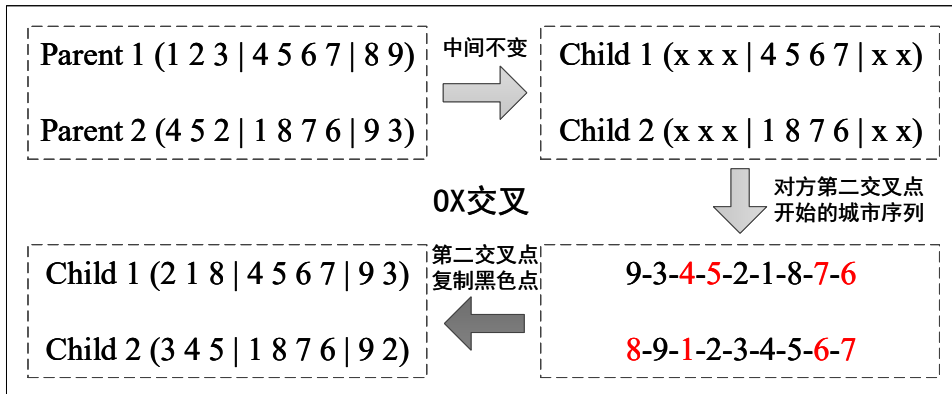
部分匹配交叉



- 'x' 标记了需交换位置
- 中间部分给出了对换关系
- Child1: 'x' 位置原有的 1 换为 4, 8 换为 5
- Child2: 'x' 位置原有的 4 换为 1, 5 换为 8



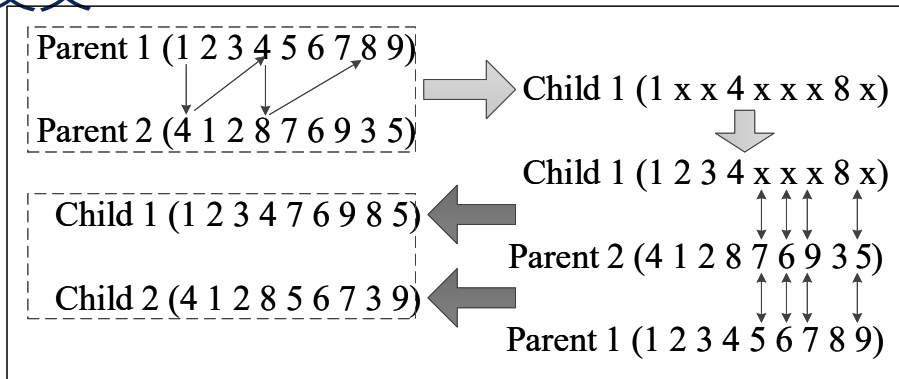
顺序交叉



交叉时需要从对方的第三部分开始按顺序复制元素，比如对于 Child1，从 Child2 的第三部分，即 9 开始，依次为 9, 3, 4, 5, 2, 1, 8, 7, 6，但要避开红色元素 (中间元素不动)

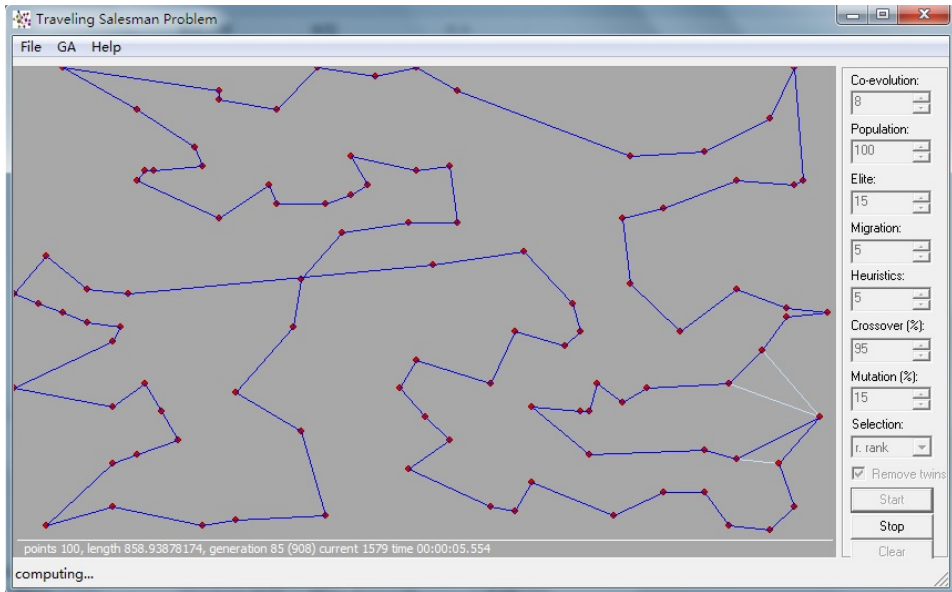


循环交叉



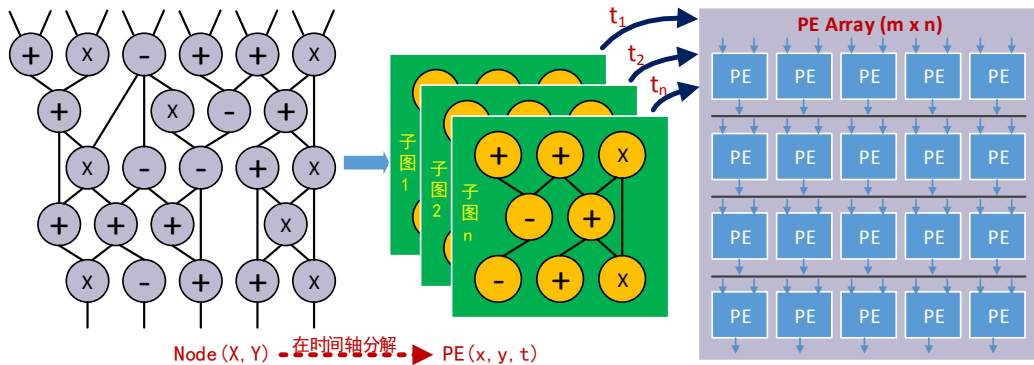
- 按照对应关系，(1,4)、(4,8)、(8,3)、(3,2)、(2,1) 对应，最后折回 1，将 1, 2, 3, 4, 8 元素不动，剩余部分进行交叉
- 对两个父个体，将 'x' 位置元素进行交换，比如 Parent1 的四个 'x' 换为 Parent2 对应位置的 7, 6, 9, 5

遗传算法解 TSP 问题示例

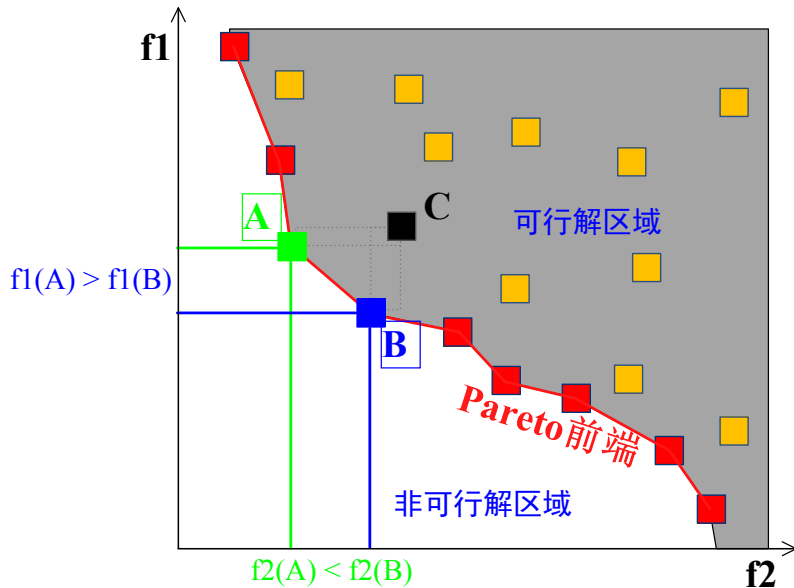




时域划分问题



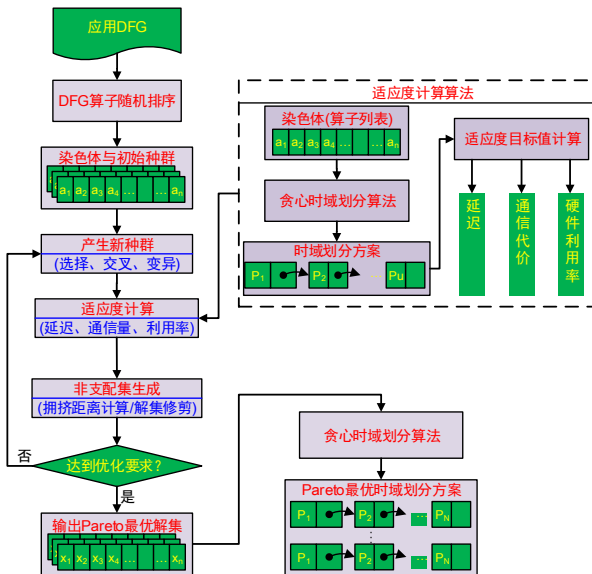
Pareto 最优



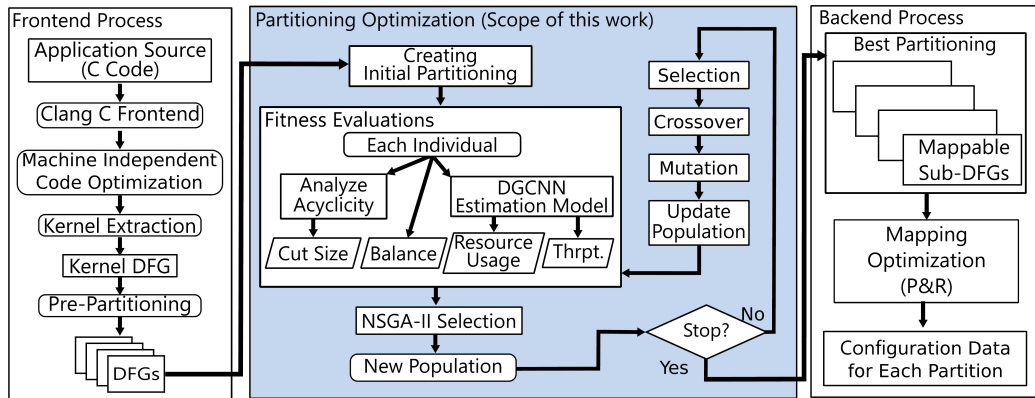


多目标时域划分的遗传算法流程

文献来源: Sheng W., et al. Pareto optimal temporal partition methodology for re-configurable architectures based on multi-objective genetic algorithm. 2012 IEEE 26th International Parallel and Distributed Processing Symposium Workshops.



前述算法改进—多目标遗传算法与深度学习结合



文献来源: Kojima T., Ohwada A., Amano H. Mapping-Aware Kernel Partitioning Method for CGRAs Assisted by Deep Learning[J]. IEEE Transactions on Parallel and Distributed Systems, 2022,33(5): 1213-1230.

IEEE 数据库中的遗传算法相关研究



☐ Journals (306)

☐ Conferences (39)

☐ Early Access Articles (17)

Search

Documents

Images (Beta)

Show

☒ All Results

☐ Subscribed Content [?](#)

☐ Open Access Only

Year



☒ Range

☐ Single Year

2020

2024

Clear

Apply

Author



Affiliation



Publication Title



Sign In to Save Your Search [?](#)



Get notified when new research is published matching your search criteria.

* Email Address

* Password

Sign In

[Forgot Password?](#) | [Create Account](#)

☐ Select All on Page

Sort By [Relevance](#)

☐ Estimation of Material Characteristics of Film-Type Noise Suppressor Using Equivalent Circuit Modeling and Genetic Algorithm [?](#)

Takahiro Mikami; Sho Muroga; Motoshi Tanaka; Yasushi Endo; Masayuki Naoe; Shuichiro Hashi; Kazushi Ishiyama
IEEE Transactions on Magnetics

Year: 2023 | Volume: 59, Issue: 11 | Journal Article | Publisher: IEEE

Cited by: Papers (1)

[?](#) Abstract [HTML](#) [?](#) [?](#)

☐ An Analog Circuit Design and Optimization System With Rule-Guided Genetic Algorithm [?](#)

Ranran Zhou; Peter Poechmueller; Yong Wang

IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

Year: 2022 | Volume: 41, Issue: 12 | Journal Article | Publisher: IEEE

Cited by: Papers (25)



提纲

1. 现代优化算法简介

2. 线性规划算法

2.1 线性规划算法简介

2.2 规划算法的专业相关应用及其求解器

3. 模拟退火算法

3.1 模拟退火算法求解旅行商问题

3.2 模拟退火算法求解套裁问题

3.3 模拟退火的专业相关应用 (以 Placement 为例)

4. 遗传算法

4.1 遗传算法数值优化

4.2 遗传算法组合优化之旅行商问题

4.3 遗传算法的专业相关应用

5. 粒子群算法

5.1 粒子群算法用于数值优化

5.2 粒子群算法的专业相关应用



粒子群算法起源

- 粒子群 (PSO: Particle Swarm Optimization) 算法起源于对鸟群觅食行为和简单社会行为的模拟
- 1995 年, 社会学家 Kennedy 和电气工程师 Eberhart 在 IEEE Neural Networks 会议上提出了 PSO 模型
- 与传统梯度优化方法不同, 粒子群等群体智能算法有如下特点:
 - 依靠概率搜索算法, 无集中控制机制, 具有更强的鲁棒性
 - 采用间接的信息交流方式, 具有良好的可扩展性
 - 具有潜在的并行性和分布式特点
 - 对问题定义的连续性和梯度信息无特殊要求
 - 算法仅涉及基本的数学操作, 易于实现

PSO 算法主要用于连续空间内优化问题的求解, 离散优化问题需要做特殊变换。应用领域包括: 电力系统、作业调度、资源配置、机器学习、聚类分析、机器人控制等

鸟群觅食





PSO 的优化思想

- 鸟群、鱼群中的个体成员能够受益于群体中其它个体在寻找食物过程中的发现和以前的经验，这种信息分享的收益远远超过了成员之间竞争导致的不利之处
- 人类同样遵循上述简单的群体聚集行为，通过不断的交互，人们总会变得更相似
- PSO 模型：
 - PSO 模型由一定数量的粒子组成，这些粒子没有质量和体积，但具有简单的社会行为
 - 粒子在解空间中移动，可通过目前位置计算适应度，当遇到优于自身经验的信息时，会更新自身的经验
 - 当群体环境改变，粒子会改变自身行为，对环境具有适应能力
 - PSO 中粒子具有学习能力，根据自身经验和群体共享经验两种信息引导其移动 (搜索) 过程



从搜索的角度理解 PSO

- PSO 算法可以认为是通过大量粒子在解空间内的移动来完成优化解的搜索 (见演示视频)
- 可认为每个粒子是问题的一个潜在解, 开始时在解空间内随机分布
- 粒子在解空间内具有速度, 因此会在解空间内移动, 从而完成搜索过程
- 粒子移动时, 会考虑自身以往所经过的最优位置 (个体自身经验), 同时会向邻域中更优粒子靠近 (群体共享经验)
- 经过连续的移动, 完成对解空间的搜索, 最优位置粒子对应问题的最优解



经典 PSO 模型

- 设问题的搜索空间为 D 维，群体规模为 N
- 粒子 i 在 D 维解空间中的位置对应问题的一个解，因此每个粒子 i 由 3 个向量描述：
 - 位置向量 $X_i = (X_{i1}, X_{i2}, \dots, X_{iD})$
 - 速度向量 $V_i = (V_{i1}, V_{i2}, \dots, V_{iD})$ ，速度向量用于移动粒子位置
 - 历史最优位置向量 pbest: $P_i = (P_{i1}, P_{i2}, \dots, P_{iD})$
- PSO 模型中粒子 i 的速度和位置按如下公式更新：

$$\begin{aligned} V_{id}(t+1) &= wV_{id}(t) + c_1r1_{id}(t)(P_{id}(t) - X_{id}(t)) \\ &\quad + c_2r2_{id}(t)(P_{gd}(t) - X_{id}(t)) \\ X_{id}(t+1) &= X_{id}(t) + V_{id}(t+1) \end{aligned}$$



经典 PSO 模型参数解析

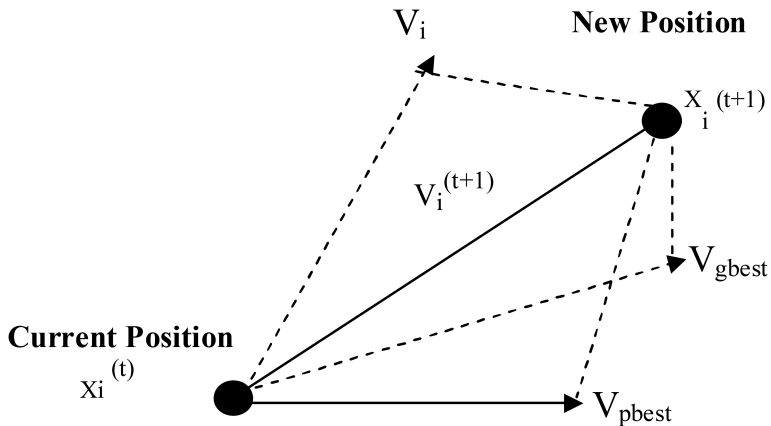
$$V_{id}(t+1) = wV_{id}(t) + c_1r1_{id}(t)(P_{id}(t) - X_{id}(t)) \\ + c_2r2_{id}(t)(P_{gd}(t) - X_{id}(t))$$

$$X_{id}(t+1) = X_{id}(t) + V_{id}(t+1)$$

- X_{id} 和 V_{id} 代表粒子 i 第 d 维的位置与速度分量, $V_{id} \leq V_{max}$
- P_{id} 表示粒子 i 的历史最佳位置, P_{gd} 为群体最佳粒子位置
- w 为惯性权重, 决定了对粒子当前速度的继承度, 使得粒子保持惯性, 增强搜索空间的能力。增大 w 利于全局搜索, 减小 w 利于在局部精细搜索
- $r1$ 和 $r2$ 为 $[0, 1]$ 区间的随机数, 称为随机因子
- $c1$ 和 $c2$ 称为加速系数, 前者为个体认知加速系数, 后者为群体认知加速系数, 决定了粒子对自身经验的学习以及对群体共享经验的学习



粒子位置和速度更新示意



粒子速度和位置的更新可视为受自身惯性、个体历史经验、群体共享经验三者合力的影响！



PSO 的优化

- 参数选择
 - 惯性权重比加速系数对算法影响更大，群体认知加速系数比个体认知加速系数更重要
 - 线性递减惯性权重可显著优化 PSO，被广泛采用
 - $c1$ 和 $c2$ 的选择：1) $c1 = c2 = 2$; 2) $c1 + c2 = 2.988$; 3) $c1 + c2 \leq 4$; 4) $c1 = 2.8, c2 = 1.3$; 5) 变系数
 - 种群规模对算法影响不大，一般取 20 ~ 60
- 邻域空间与拓扑结构研究
 - 局部版 PSO：复杂问题应采用小邻域，简单问题采用大邻域
 - 全息 PSO：抛弃自身历史最好解和全局历史最好解，利用当前粒子的所有邻居信息更新位置和速度
 - 解向量分组，各自用一子 PSO 算法搜索
- 与其它技术融合：与模拟退火、遗传算法、免疫算法等结合



经典 PSO 算法流程

- 1) 设置参数：惯性权重、加速系数、种群规模、速度/位置取值范围、最大迭代次数或适应度值误差限
- 2) 随机初始化种群，主要是粒子的初始位置、初始速度
- 3) 按目标函数评价各粒子初始适应度值，将适应度值最好者赋给 $gbest$
- 4) 更新各粒子速度，需注意新速度要进行限幅处理
- 5) 更新各粒子位置，需注意新位置要进行限幅处理
- 6) 按目标函数重新评价各粒子适应度
- 7) 更新各粒子历史最好解 $pbest$
- 8) 更新种群最好解 $gbest$
- 9) 若已完成设定迭代次数或适应度误差限满足要求，结束；否则返回步骤 4) 继续搜索



经典 PSO 算法流程图

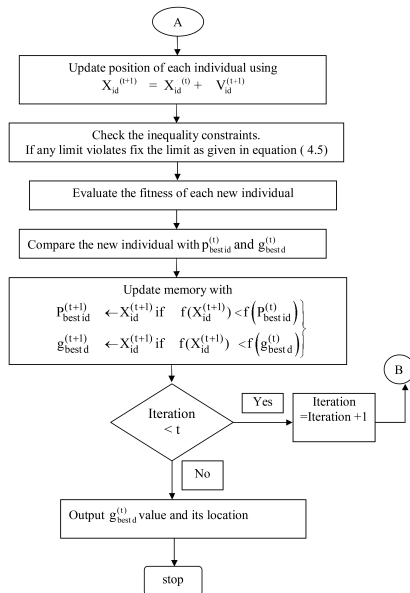
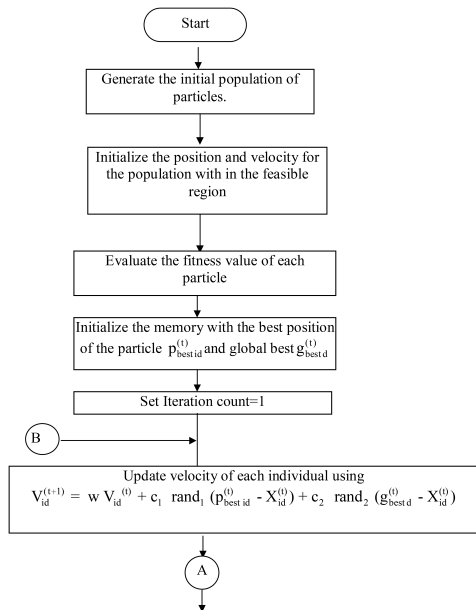


Figure 4.2 (Continued)

PSO 算法 DEMO 视频



PSO 算法 DEMO 视频

PSO 数值优化 —— Particle 数据结构的设计



仍以求 $f(x) = x \sin(10\pi \cdot x) + 2.0$ $x \in [-1, 2]$ 最大值为例!

```
private:
    std::vector<double> _X;
    std::vector<double> _V;
    double _fitness;
    double _pbest_fitness;
    std::vector<double> _pbest;
```



Particle 的位置和速度更新

```
void fly(unsigned int epoch, double w, ...) {  
    for (size_t i = 0; i < _V.size(); i++)  
    {  
        _V[i] = w * _V[i]  
            + c1 * rand01()*(_pbest[i] - _X[i])  
            + c2 * rand01()*(gbest[i] - _X[i]);  
        if (_V[i] < -maxV) _V[i] = -maxV;  
        else if (_V[i] > maxV) _V[i] = maxV;  
        _X[i] += _V[i];  
        if (_X[i] < lb[i]) _X[i] = lb[i];  
        if (_X[i] > hb[i]) _X[i] = hb[i];  
    }  
    evaluate();  
    updatePBest();  
}
```




PSO 的迭代

```
void optimize() {  
    for (int i = 0; i < _T; i++) {  
        if (i % 100 == 0 && i > 0)  
            std::cout << "epoch " << i << std::endl;  
        double w = _W_0 - (_W_0 - _W_T)*i / (_T - 1);  
        for (auto& particle : _particles) {  
            particle.fly(i, w, _c1, _c2, _MAX_V, _lb, _hb,  
                _gbest->getPBest());  
            if (isBetter(particle.getFitness(), _gbest->  
                getPBestFitness()))  
                _gbest = &particle;  
        }  
    }  
}
```



PSO 参数设置

```
private:
unsigned int _N = 50; // Number of particles
unsigned int _nD = 1; // Number of dimensions
unsigned int _T = 1000; // Maximum number of epochs
// Inertia weight at  $t=0$  ( $W_0$ ) and  $t=T$  ( $W_T$ )
double _W_0 = 0.9;
double _W_T = 0.4;
double _MAX_V = 1.5;
std::vector<double> _lb; // lower bounds
std::vector<double> _hb; // higher bounds
// Cognitive parameter( $c1$ ) and social parameter( $c2$ )
double _c1 = 2.0;
double _c2 = 2.0;
std::vector<Particle> _particles; // the particles
Particle *_gbest = nullptr; // global best particle
```



PSO 函数优化的运行截图

The screenshot shows a C++ IDE with a file named `main.cpp` open. The code implements a Particle Swarm Optimization (PSO) algorithm. It includes a `for` loop that iterates over a vector `_V`. Inside the loop, it updates the velocity `_V[i]` based on the current position `_X[i]` and the best-known position `_pbest[i]` and `gbest[i]`. It also updates the position `_X[i]` based on the new velocity. The code includes boundary checks for velocity and position. A terminal window on the right shows the output of the program, displaying the results for 100 epochs, the final fitness value, and the optimized scheme.

```
for (size_t i = 0; i < _V.size(); i++)
{
    _V[i] = w * _V[i]
        + c1 * rand01()*(_pbest[i] - _X[i])
        + c2 * rand01()*(gbest[i] - _X[i]);
    if (_V[i] < -maxV)
        _V[i] = -maxV;
    else if (_V[i] > maxV)
        _V[i] = maxV;
    _X[i] += _V[i];
    if (_X[i] < lb[i]) _X[i] = lb[i];
    if (_X[i] > hb[i]) _X[i] = hb[i];
}
```

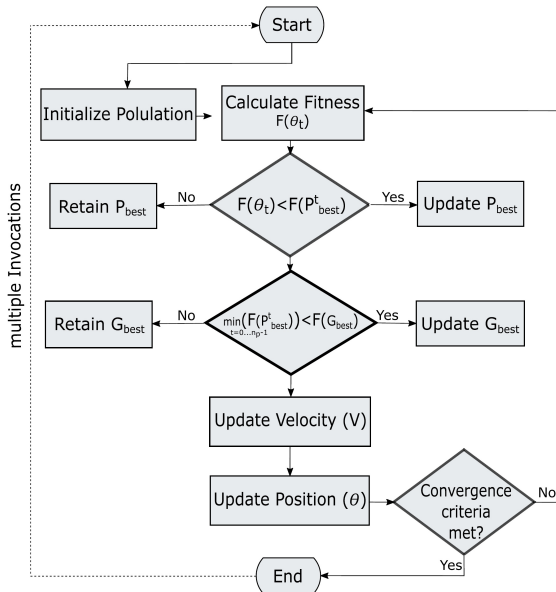
```
D:\devel\optimization\psd\psd\6...
epoch 100
epoch 200
epoch 300
epoch 400
epoch 500
epoch 600
epoch 700
epoch 800
epoch 900
Fitness: 3.85027
The optimized scheme:
x0=1.85055,
```

实际运行结果显示：PSO 算法比 GA 算法优化效果好，速度更快！



PSO 算法用于 SNN 映射优化

在 TPDS2022 论文: Endurance-Aware Mapping of Spiking Neural Networks to Neuromorphic Hardware 中, PSO 算法用来将 Cluster 映射到神经形态硬件的 Tile 上!



IEEE 数据库中的 PSO 算法相关研究



Year

Range

Single Year

2020

2024

Clear

Apply

Author

Affiliation

Publication Title

Publisher

Publication Topics

- ☐

A Speculative Divide-and-Conquer Optimization Method for Large Analog/Mixed-Signal

Circuits: A High-Speed FFE SST Transmitter Example

Kwangmin Kim; Hyoseok Song; Byeongcheol Lee; Byungsub Kim

IEEE Transactions on Very Large Scale Integration (VLSI) Systems

Year: 2023 | Volume: 31, Issue: 5 | Journal Article | Publisher: IEEE

Cited by: Papers (2)

Abstract

HTML

☐

Design of Sequential Load Modulation Balance Amplifier Using Multiobjective Particle Swarm Algorithm

Zhiming Fan; Jiajun Huang; Jialin Cai

IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

Year: 2024 | Volume: 43, Issue: 8 | Journal Article | Publisher: IEEE

Abstract

HTML

☐

Flexible Inverse Design of Microwave Filter Customized on Demand With Wavelet Transform Deep Learning

Kuiwen Xu; Jianguo Wang; Jialin Cai; Xuetao Ma; Qinyi Lv; Shichang Chen; Jie Liu; Jun Liu

IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

Year: 2024 | Early Access Article | Publisher: IEEE

Abstract

☐

Microwave System of Transverse Output for a High-Power W -Band Gyro-TWT

Alexandr A. Bogdashov; Sergey V. Samsonov

IEEE Transactions on Electron Devices

7

105 / 107



四种优化算法对比

- 线性规划算法约束较为复杂，一般用软件包进行求解
- 模拟退火模拟了金属退火过程，通过以概率接受劣化解跳出局部最优
- 遗传算法模拟了生物进化过程，通过以概率选择、交叉和变异操作跳出局部最优
- 粒子群算法模拟了鸟群觅食过程，通过分散粒子各自搜索、粒子间互相学习、粒子惯性飞越完成解空间搜索并跳出局部最优
- 这些算法各具特点，但粒子群算法求解组合优化问题不如模拟退火和遗传算法方便，而且经典 PSO 的最大速度界限难以根据经验估计

小结



本节课重点：

- 现代优化算法的种类（了解）
- 数值优化问题的优化求解方法（重点）
- 组合优化问题的优化求解方法（重点）